

Name	RAVIVARMAN S
BITS ID	2025DA0430
Course Code	DSECLZG565
Course Name	Machine Learning (S2-25)

Assignment 2

Predict Students' Dropout and Academic Success

Model Comparison & Streamlit Deployment

1. GitHub Repository Link

Repository containing the complete source code, requirements.txt, README.md, and test data (CSV):

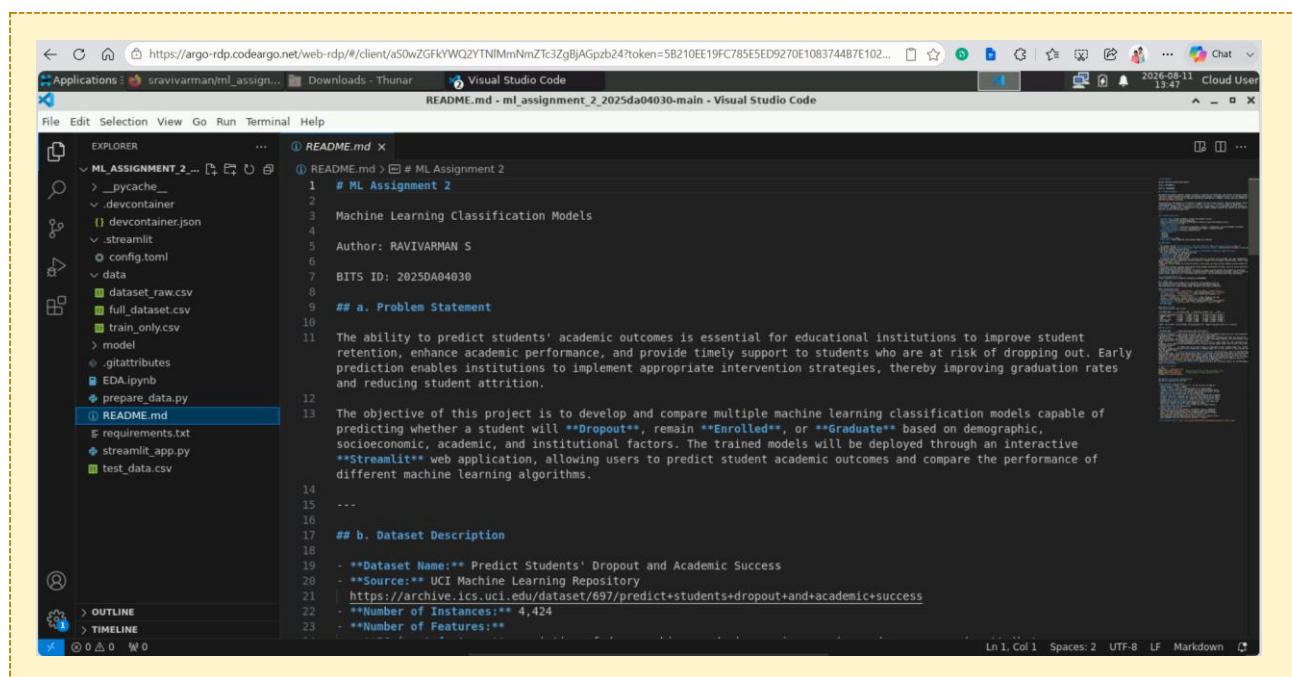
https://github.com/sravivarma/ml_assignment_2_2025da04030

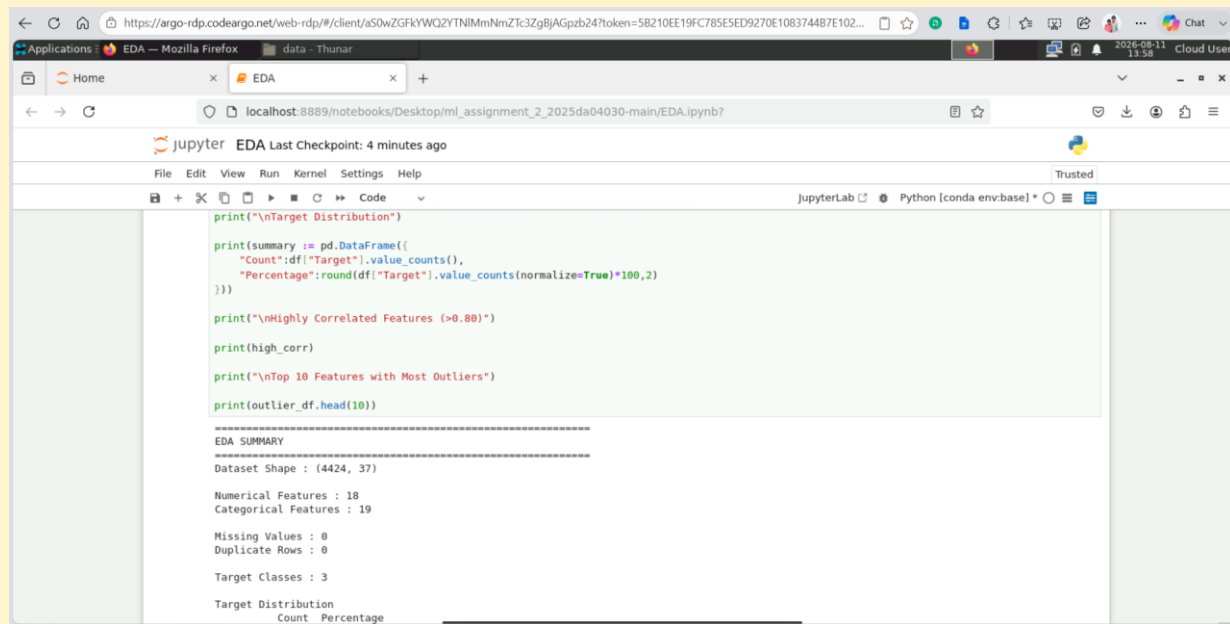
2. Live Streamlit App Link

Deployed using Streamlit Community Cloud:

<https://mlassignment22025da04030-pwaah8udxgjlyztpysgvbz.streamlit.app/>

3. Screenshot — BITS Virtual Lab Execution





```

print("\nTarget Distribution")

print(summary := pd.DataFrame({
    "Count":df["Target"].value_counts(),
    "Percentage":round(df["Target"].value_counts(normalize=True)*100,2)
}))

print("\nHighly Correlated Features (>0.80)")

print(high_corr)

print("\nTop 10 Features with Most Outliers")

print(outlier_df.head(10))

=====
EDA SUMMARY
=====
Dataset Shape : (4424, 37)

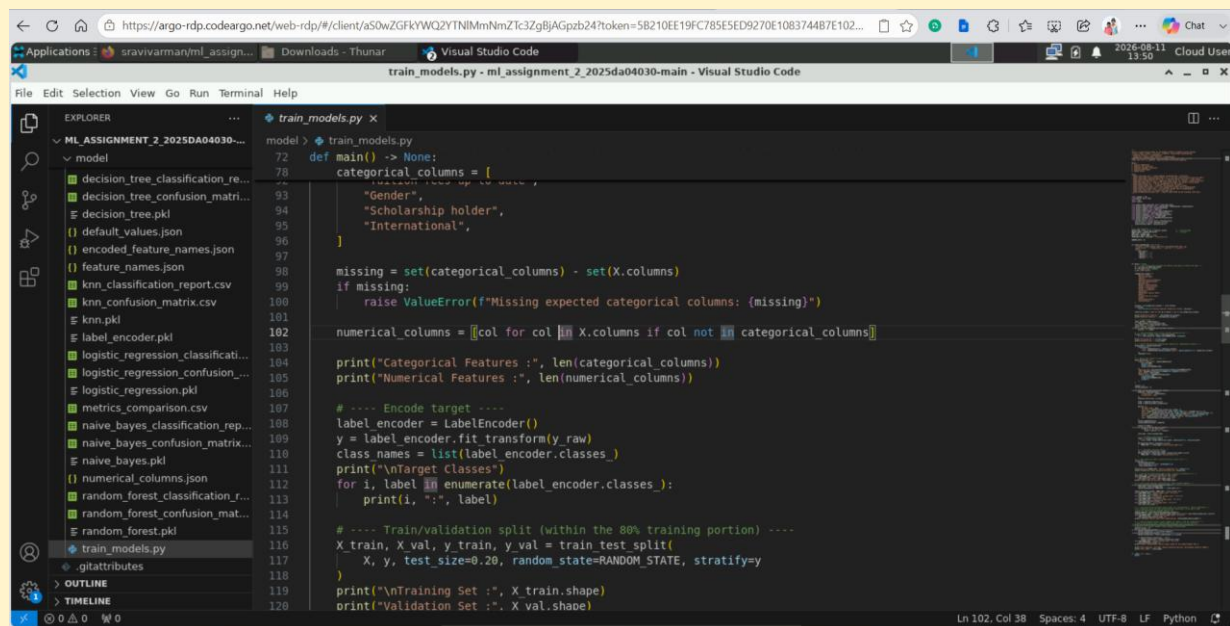
Numerical Features : 18
Categorical Features : 19

Missing Values : 0
Duplicate Rows : 0

Target Classes : 3

Target Distribution
Count Percentage

```



```

def main() -> None:
    categorical_columns = [
        "Gender",
        "Scholarship holder",
        "International",
    ]

    missing = set(categorical_columns) - set(X.columns)
    if missing:
        raise ValueError(f"Missing expected categorical columns: {missing}")

    numerical_columns = [col for col in X.columns if col not in categorical_columns]

    print("Categorical Features :", len(categorical_columns))
    print("Numerical Features :", len(numerical_columns))

    # ---- Encode target ----
    label_encoder = LabelEncoder()
    y = label_encoder.fit_transform(y_raw)
    class_names = list(label_encoder.classes_)
    print("\nTarget Classes")
    for i, label in enumerate(label_encoder.classes_):
        print(i, ":", label)

    # ---- Train/validation split (within the 80% training portion) ----
    X_train, X_val, y_train, y_val = train_test_split(
        X, y, test_size=0.20, random_state=RANDOM_STATE, stratify=y
    )
    print("\nTraining Set :", X_train.shape)
    print("\nValidation Set :", X_val.shape)

```

4. GitHub README Content

a. Problem Statement

The ability to predict students' academic outcomes is essential for educational institutions to improve student retention, enhance academic performance, and provide timely support to students who are at risk of dropping out. Early prediction enables institutions to implement appropriate intervention strategies, thereby improving graduation rates and reducing student attrition. The objective of this assignment is to develop and compare multiple machine learning classification models capable of predicting whether a student will **Dropout**, remain **Enrolled**, or **Graduate** based on demographic, socioeconomic, academic, and institutional factors. The trained models will be deployed through an interactive **Streamlit** web application, allowing

users to predict student academic outcomes and compare the performance of different machine learning algorithms.

b. Dataset Description

- **Dataset Name:** Predict Students' Dropout and Academic Success
- **Source:** UCI Machine Learning Repository
<https://archive.ics.uci.edu/dataset/697/predict+students+dropout+and+academic+success>
- **Number of Instances:** 4,424
- **Number of Features:**
 - 36 input features consisting of demographic, academic, socioeconomic, and macroeconomic attributes.
 - 1 target variable (`Target`) representing the student's academic outcome.
- **Classification Type:** Multiclass Classification
- **Target Classes:**
 - Dropout
 - Enrolled
 - Graduate
- **Missing Values:** None
- **Data Type:** Mixed (Numerical and Encoded Categorical Features)

The dataset comfortably exceeds the assignment's minimum requirements (≥ 500 instances, ≥ 12 features). Because the target has 3 classes, AUC is computed as a One-vs-Rest macro average, and Precision/Recall/F1 use a weighted average across classes.

EDA Summary

- The dataset contains **4,424 records**, **36 input features**, and **one target variable (`Target`)**.
- The dataset contains **no missing values** and **no duplicate records**, indicating that the data is clean and suitable for machine learning analysis.
- The target variable consists of **three classes: Graduate, Dropout, and Enrolled**.
- The class distribution is reasonably balanced:
 - **Graduate:** 2,209 records (49.93%)
 - **Dropout:** 1,421 records (32.12%)
 - **Enrolled:** 794 records (17.95%)
- The dataset contains a combination of continuous numerical variables (such as grades, age, GDP, unemployment rate, and inflation rate) and categorical variables encoded as integers (such as gender, marital status, course, scholarship holder, and debtor status).

- Numerical features exhibit varying distributions, with several variables showing skewness and the presence of outliers.
- Correlation analysis indicates relationships among academic performance variables, while no severe multicollinearity is observed across the numerical features.
- Based on the exploratory data analysis, the dataset is clean, well-structured, and suitable for multiclass classification. Appropriate preprocessing, including handling encoded categorical variables, feature scaling for distance-based algorithms, and a stratified train-test split, will be performed before model development.

c. GitHub Repository Link

https://github.com/sravivarman/ml_assignment_2_2025da04030

d. Models Used

All 5 models were trained on the same 80/20 train/validation split, using the shared preprocessing pipeline described above and a label-encoded target (Dropout=0, Enrolled=1, Graduate=2).

Model Configuration Notes

- **Logistic Regression:** solver="lbfgs", max_iter=5000, random_state=42.
- **Decision Tree:** random_state=42; all other hyperparameters (e.g. max_depth, min_samples_split, criterion="gini") left at scikit-learn defaults, so the tree grows until its leaves are pure.
- **kNN:** n_neighbors=5, weights="distance" — closer neighbors get more influence, which improved results slightly over uniform weighting.
- **Naive Bayes:** GaussianNB() with default hyperparameters (no tuning applied).
- **Random Forest:** n_estimators=300, random_state=42, n_jobs=-1 (uses all CPU cores).

Comparison Table (sorted by Accuracy, best first)

ML Model Name	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.7782	0.8928	0.7674	0.7782	0.7701	0.6331
Random Forest	0.7726	0.8759	0.7519	0.7726	0.7533	0.6219
kNN	0.7316	0.8229	0.7181	0.7316	0.7143	0.5515
Decision Tree	0.6949	0.7358	0.6925	0.6949	0.6929	0.5049
Naive Bayes	0.2556	0.6883	0.2966	0.2556	0.1846	0.1284

(AUC = One-vs-Rest, macro-averaged; Precision/Recall/F1 = weighted average across the 3 classes)

Observations

ML Model Name	Observation about model performance
Logistic Regression	Best overall performer. The linear decision boundary generalizes well once categorical codes are properly one-hot encoded instead of being treated as ordinal numbers.
Random Forest	Second-best, close behind Logistic Regression. Averaging many trees over the one-hot + scaled feature set captures non-linear interactions (e.g. how curricular performance combines with socio-economic factors) while staying robust to the high-dimensional categorical space.
kNN	Distance-weighted voting (`weights="distance"`) gave a small but real improvement over uniform weighting. Still capped by the high-dimensional, mostly-binary one-hot feature space, which dilutes the meaning of Euclidean distance.
Decision Tree	A single tree can now split cleanly on individual one-hot category flags (e.g. "Course = Nursing?") instead of arbitrary numeric thresholds on category codes, but it still overfits to the training split more than the ensemble model.
Naive Bayes	Collapsed sharply after one-hot encoding (25.6% accuracy). This is a well-known Naive Bayes failure mode: GaussianNB assumes every feature is continuous and normally distributed, but one-hot encoding expanded the feature space to 232 mostly-binary (0/1) columns. Many of these have near-zero variance for rare categories, which distorts GaussianNB's per-feature Gaussian likelihood estimates and causes it to over-predict a single class. A CategoricalNB/BernoulliNB treatment of the categorical block would likely fix this — it demonstrates why the choice of Naive Bayes variant must match the data type.
Overall Winner	Logistic Regression — highest Accuracy, AUC, Precision, Recall, F1, and MCC among all 5 models.

How to Run Locally

```
pip install -r requirements.txt
```

```
python data/prepare_data.py    # generates the CSVs (already included in repo)
```

```
python model/train_models.py   # trains models (already included in repo)
```

```
streamlit run app.py
```

Deployment on Streamlit Community Cloud

The app is organized into two tabs:

- **Bulk Evaluation (CSV Upload)**
 - **CSV upload** — upload `test\_data.csv` (or any CSV with the same 36 feature columns + `Target` label)
 - **Model selection dropdown** — choose among all 5 trained models (sidebar, shared across both tabs; defaults to the best-performing model)
 - **Evaluation metrics display** — Accuracy, AUC, Precision, Recall, F1, MCC shown live for the selected model on the uploaded data
 - **Confusion matrix & classification report** — 3×3 confusion matrix heatmap (Dropout / Enrolled / Graduate) and a full per-class classification report
 - **Domain-specific interpretation** — separately flags the count and rate of students who **actually dropped out but were predicted "Graduate"** — the costliest error type for an early-intervention use case, since these are exactly the students who wouldn't get flagged for support
 - **All-model comparison table + chart** — see how all 5 models perform side-by-side on the same uploaded data
- **Single Student Prediction**
 - A manual entry form for one student's key details (course, attendance, age, scholarship/tuition/debtor status, semester grades and approved units); all other features default to the median/mode "typical student" value from the training data (model/default_values.json)
 - Live prediction with a probability bar chart across all 3 classes, plus a plain-language flag when the predicted outcome is "Dropout"

Live Links

GitHub Repository:

https://github.com/sravivarman/ml_assignment_2_2025da04030

Live Streamlit App:

<https://mlassignment22025da04030-pwaah8udxgjlyztpysgvbz.streamlit.app/>

Final Submission Checklist

- ☒ GitHub repo link works
- ☒ Streamlit app link opens correctly
- ☒ App loads without errors
- ☒ All required features implemented
- ☒ README.md updated and added in the submitted PDF.